FitTracker

Executive Technical Architecture Specification, SRS Standard, and Mathematical Model of Progressive Overload

Lead Software Architect & Mobile Tech Lead

August 14, 2026

Abstract

This document serves as the comprehensive software engineering specification for the mobile application **FitTracker** [1, 2]. Engineered under a strict *Local-First* architecture [3], FitTracker automates resistance training progression through mathematical 1RM ($1 Repetition Maximum$) estimations [4, 5] and RPE/RIR autoregulation [6, 7]. Adhering to the IEEE 830 SRS standard [8], this specification details functional and non-functional requirements, UML use cases, SQLite relational DDL schema, UI design token system, automated test suite verification, and deployment readiness.

Contents

1 Introduction and Objectives

1.1 Context and Rationale

In resistance training and muscle hypertrophy, the principle of **progressive overload** serves as the core foundation for driving long-term physiological adaptations. However, the vast majority of mobile applications on the market currently suffer from two critical limitations:

1. Strict reliance on Internet connectivity and cloud infrastructure (*Cloud-First*), which degrades the user experience in gyms with weak coverage or underground facilities.
2. Lack of deterministic automatic progressive overload algorithms based on physiological variables and quantitative metrics such as estimated 1RM and the RPE/RIR scale (*Rating of Perceived Exertion / Repetitions in Reserve*).

FitTracker is designed to solve both limitations through a **Local-First** mobile architecture powered by an embedded SQLite engine and an adaptive algorithmic progression model.

1.2 System Objectives

1.2.1 General Objective

Design, architect, and implement a cross-platform mobile application (React Native + Expo in TypeScript) characterized by 100% offline operation and Clean Architecture, capable of calculating and automatically increasing workload and volume for athletes based on historical performance.

1.2.2 Specific Objectives

- Implement an ultra-fast local persistence layer using SQLite, ensuring zero network latency and fault tolerance.
- Develop the mathematical 1RM ($1 Repetition Maximum$) estimation module by averaging classic Epley and Brzycki models with RIR correction.
- Provide a modular system architecture strictly separated into Domain, Use Cases, Repositories, and UI layers.
- Deliver formal academic technical documentation for engineering audit and research.

1.3 Acknowledgments and Data Sources

We formally acknowledge developer **Hasan Yıldırım** for creating and maintaining the open-source repository `exercises-dataset`. His structured JSON dataset provides the complete exercise database, metadata, and media elements powering the offline local seeding system in our persistence layer.

1.4 Artificial Intelligence Assistance Statement

This software project, including system architectural design, mathematical progression model formulation, embedded SQLite persistence layer, TypeScript source code, Jest unit test suite, and formal LaTeX technical documentation, was developed in pair-programming collaboration with **Antigravity**, an AI Coding Assistant developed by Google DeepMind.

2 Software Requirements Specification (SRS)

This specification adapts the IEEE 830 standard for offline-first mobile systems.

2.1 Functional Requirements (FR)

Table 1: System Functional Requirements Specification

Code	Requirement Name	Technical Description & Acceptance Criteria
FR-01	Exercise Catalog Management	Search and filter +1,500 exercises by muscle group (Chest, Back, Legs, Shoulders, Biceps, Triceps) and equipment type, plus custom exercise creation.
FR-02	Routine Template Builder	Create, edit, organize, and launch pre-configured workout routine templates with custom target reps, sets, and order.
FR-03	Live Set Logging & Tagging	Log weight (kg), reps (r), and RPE/RIR (R) in real-time, tagging sets as Warmup (`[Calent.]`) or Effective (`[Efectiva]`).
FR-04	Automated Overload Engine	Compute recommended load (w_{rec}) and reps (r_{rec}) for upcoming sessions using RPE/RIR autoregulation rules.
FR-05	1RM Estimation & Ranks	Calculate real-time estimated 1RM via Epley formula and map strength performance to rank badges (Beginner, Intermediate, Advanced, Elite).
FR-06	Rest-Tolerant Streak Engine	Track consistency streak with an automated 3-day rest buffer allowance before resetting the streak counter.
FR-07	Profile Setup & FitMojis	Setup user profile (age, sex, height, weight, level) and enforce FitMojis animal avatar selection with random fallback.
FR-08	Dual-Language Localization	Seamlessly translate UI controls, exercise catalog, profile settings, and FitMojis labels between Spanish (ES) and English (EN).
FR-09	Background Notification Timer	Push live notifications with workout stopwatch duration, rest countdowns, and active exercise details when app is minimized.
FR-10	Native Line Chart Analytics	Render real-time 2D progression charts per exercise natively without third-party chart package dependencies.
FR-11	Local Data Backup & Export	Export full workout database in JSON format from session history view for data portability and user backup.

2.2 Non-Functional Requirements (NFR)

Table 2: System Non-Functional Requirements Specification

Code	Requirement Name	Metric / Acceptance Criteria
NFR-01	100% Local-First Operability	100% of read, write, and analytics operations execute locally in SQLite without cloud dependencies or internet connection.
NFR-02	Low Latency Database Writes	Set insertion and batch volume updates in SQLite must complete in < 30 ms on standard Android and iOS devices.
NFR-03	OLED Power Efficiency	Pitch black base theme (`#000000`) with contrast ratio $> 15:1$, minimizing display energy consumption on OLED screens.
NFR-04	Cross-Platform Code Base	Single unified React Native SDK 54 TypeScript codebase executing uniformly across Android and iOS without platform branches.
NFR-05	ACID Data Reliability	SQLite WAL (Write-Ahead Logging) journaling ensures zero data corruption during abrupt application termination or battery drain.
NFR-06	Safe Area Ergonomics	Dynamic status bar and bottom gesture inset offsets preventing UI clipping on physical notches and system navigation bars.
NFR-07	Zero Telemetry Privacy	Absolute privacy guarantee with zero external analytics tracking, device isolation, and local database encryption.

3 UML Use Cases Specification

3.1 System Actors

- **Athlete / End User:** Individual executing workout routines, logging sets, managing rest timers, and analyzing overload analytics offline.

3.2 System Use Case Diagram

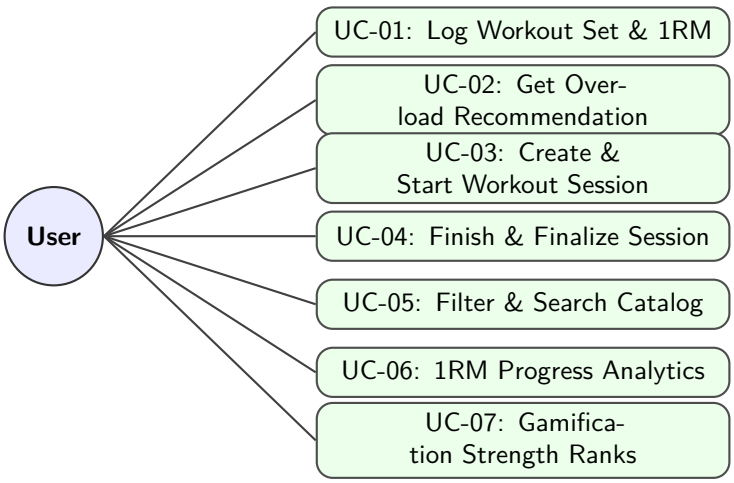

Figure 1: UML Use Case Overview for FitTracker Domain Architecture.

3.3 Detailed Specifications

3.3.1 UC-01: Log Workout Set and Compute Progression

3.3.2 UC-02: Get Automatic Overload Recommendation

Use Case	UC-01: Log Workout Set and Compute Automatic Progression
Primary Actor	Athlete / End User
Preconditions	At least one exercise exists in the catalog and an active workout session is started.
Main Flow	1. The user selects the current exercise in the active session. 2. The system displays target weight (kg) and reps calculated by UC-02. 3. The user inputs load, completed reps, and RPE ($6.0 - 10.0$). 4. The user confirms set registration. 5. The system calculates estimated 1RM, persists the set to SQLite, and updates session effective volume.
Alternative Flow	*3a. User marks set as Warmup Set:* - System records the set but excludes it from effective volume and 1RM calculation.
Postconditions	Set recorded in SQLite with calculated 1RM.

Use Case	UC-02: Get Automatic Overload Recommendation
Primary Actor	Athlete / End User
Preconditions	Exercise selected from catalog.
Main Flow	1. System fetches last working sets logged for the exercise. 2. System calculates average RIR and completion rate against target rep range. 3. System applies load rule ($+2.5\,kg$ compound / $+1.25\,kg$ isolation / $MAINTAIN$ / $DELOAD$). 4. System presents recommended load and target reps with mathematical justification.
Postconditions	Recommendation generated deterministically without side effects.

3.3.3 UC-03: Create & Start Workout Session

Use Case	UC-03: Create & Start Workout Session
Primary Actor	Athlete / End User
Preconditions	Application initialized with SQLite schema.
Main Flow	1. User enters session title (e.g., "Leg Day - Heavy") and optional notes. 2. System generates unique session identifier and records start timestamp. 3. System transitions session state to active.
Postconditions	New session record persisted in SQLite database.

3.3.4 UC-04: Finish & Finalize Workout Session

Use Case	UC-04: Finish & Finalize Workout Session
Primary Actor	Athlete / End User
Preconditions	Active workout session in progress.
Main Flow	1. User taps "Finish Workout". 2. System calculates session duration, total tonnage, and total effective volume. 3. System identifies any Personal Records (PRs) set during the session. 4. System updates session metadata and displays workout summary.
Postconditions	Session state marked complete; summary metrics stored.

3.3.5 UC-05: Filter & Search Exercise Catalog

3.3.6 UC-06: Get Exercise Historic Metrics & 1RM Evolution

3.3.7 UC-08: Complete Profile Setup & FitMojis Avatar Selection

Use Case	**UC-05: Filter & Search Exercise Catalog**
Primary Actor	Athlete / End User
Preconditions	SQLite exercise table populated with dataset.
Main Flow	1. User types search query or selects filter parameters (muscle group, equipment, type). 2. System executes SQL query with substring matching and filter constraints. 3. System returns sorted list of matching exercises.
Postconditions	Filtered exercise list displayed to user.

Use Case	**UC-06: Get Exercise Historic Metrics & 1RM Evolution**
Primary Actor	Athlete / End User
Preconditions	Exercise selected by user.
Main Flow	1. System queries SQLite for all historical sets of the target exercise. 2. System aggregates peak estimated 1RM per session date. 3. System calculates historical max weight, max volume, and total sets logged. 4. System returns time-series data suitable for charting.
Postconditions	Analytics DTO generated for visual rendering.

Use Case	**UC-08: Complete Profile Setup & FitMojis Avatar Selection**
Primary Actor	Athlete / End User
Preconditions	Application launched for the first time without an active profile.
Main Flow	1. Athlete fills in body metrics (age, sex, height, weight, experience level, language). 2. Athlete selects a FitMoji avatar character (Lion, Bear, Panther, Eagle). 3. Athlete confirms profile creation. 4. System persists profile record to SQLite and initializes dashboard.
Alternative Flow	*2a. Athlete omits avatar selection:* - System assigns a random FitMoji key (`lion`, `bear`, `panther`, `eagle`) as fallback.
Postconditions	Athlete profile saved; onboarding state completed.

3.3.8 UC-09: Export Database Telemetry JSON

Use Case	**UC-09: Export Database Telemetry JSON**
Primary Actor	Athlete / End User
Preconditions	SQLite database contains logged sessions and sets.
Main Flow	1. Athlete selects ”Export JSON” from session history interface. 2. System serializes all SQLite tables into a structured JSON string. 3. System triggers OS native share/save prompt for data backup.
Postconditions	JSON backup file exported; zero data modified in database.

4 Architecture and Database Design

4.1 System Architecture (Clean Architecture)

FitTracker implements Clean Architecture structured by functional feature modules (*Feature-First*).

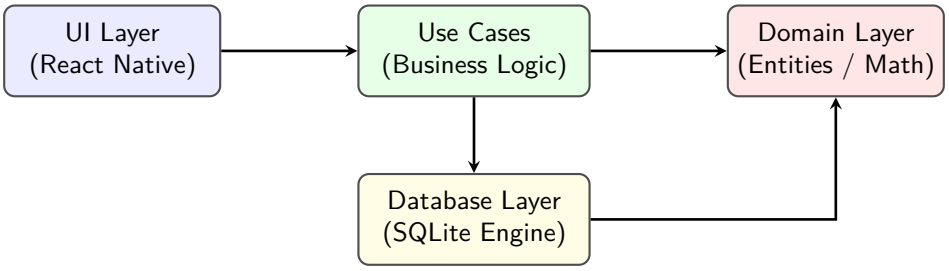

Figure 2: Layer Dependency Diagram in Clean Architecture.

4.2 Entity-Relationship Model and SQLite Schema

```sql
-- Exercises Table
CREATE TABLE exercises (
    id TEXT PRIMARY KEY NOT NULL,
    name TEXT NOT NULL,
    category TEXT NOT NULL,
    type TEXT NOT NULL,
    equipment TEXT NOT NULL,
    gif_url TEXT,
    instructions TEXT
);

-- Workout Sessions Table
CREATE TABLE workout_sessions (
    id TEXT PRIMARY KEY NOT NULL,
    name TEXT NOT NULL,
    date INTEGER NOT NULL,
    notes TEXT
);

-- Logged Exercise Sets Table
CREATE TABLE exercise_sets (
    id TEXT PRIMARY KEY NOT NULL,
    session_id TEXT NOT NULL,
    exercise_id TEXT NOT NULL,
    set_order INTEGER NOT NULL,
    weight_kg REAL NOT NULL,
    reps INTEGER NOT NULL,
    rpe REAL NOT NULL,
    is_warmup INTEGER NOT NULL DEFAULT 0,
    estimated_1rm REAL NOT NULL,
    rest_seconds INTEGER NOT NULL DEFAULT 0,
    FOREIGN KEY(session_id) REFERENCES workout_sessions(id) ON DELETE CASCADE,
    FOREIGN KEY(exercise_id) REFERENCES exercises(id) ON DELETE CASCADE
);

-- Athlete User Profile Table
CREATE TABLE user_profile (
    id TEXT PRIMARY KEY NOT NULL,
    name TEXT NOT NULL,
    age INTEGER NOT NULL,
    sex TEXT NOT NULL,
```

```
    height_cm REAL NOT NULL,
    body_weight_kg REAL NOT NULL,
    experience_level TEXT NOT NULL,
    language TEXT NOT NULL DEFAULT 'es',
    avatar_key TEXT
);
```

Listing 1: SQLite Relational DDL Schema

5 Automatic Progressive Overload Algorithm

5.1 Mathematical Formulations for 1RM Estimation

To quantify relative strength in a set of r repetitions executed with load w (expressed in kg) at a perceived exertion of $RPE \in [6.0, 10.0]$, we first calculate equivalent Repetitions in Reserve (RIR):

$$RIR = 10.0 - RPE \tag{1}$$

Estimated repetitions to absolute failure $r_{\max}$ are calculated as:

$$r_{\max} = r + RIR \tag{2}$$

We use a weighted combination of classic non-linear models by **Epley** and **Brzycki**:

1. **Epley Formula:**
$$1RM_{\text{Epley}} = w \cdot \left(1 + \frac{r_{\max}}{30}\right) \tag{3}$$
2. **Brzycki Formula:**
$$1RM_{\text{Brzycki}} = w \cdot \frac{36}{37 - r_{\max}} \tag{4}$$

The final estimated set 1RM ($1RM_{\text{est}}$) is computed by averaging both models:

$$1RM_{\text{est}} = \frac{1RM_{\text{Epley}} + 1RM_{\text{Brzycki}}}{2} \tag{5}$$

5.2 Effective Volume and Automatic Progressive Overload

The **Total Effective Volume** ($V_{\text{effective}}$) of a session for a given exercise, considering working sets only ($is\_warmup = 0$), is expressed as:

$$V_{\text{effective}} = \sum_{i=1}^{N} w_i \cdot r_i \cdot \alpha(RPE_i) \tag{6}$$

where $\alpha(RPE_i)$ represents the fatigue weighting factor:

$$\alpha(RPE) = \begin{cases} 1.0 & \text{if } RPE \geq 8.0 \\ 0.85 & \text{if } 7.0 \leq RPE < 8.0 \\ 0.70 & \text{if } RPE < 7.0 \end{cases} \tag{7}$$

5.3 Next Session Load Progression Rule

The recommended load w_{next} for the subsequent session is adjusted based on session average RIR_{avg} and target repetition range $[R_{\min}, R_{\max}]$ completion:

$$w_{\text{next}} = \begin{cases} w_{\text{current}} + \Delta w & \text{if } r_i \geq R_{\max}\ \forall i \quad \text{and} \quad RIR_{\text{avg}} \geq 1.5 \\ w_{\text{current}} & \text{if } R_{\min} \leq r_i < R_{\max} \quad \text{or} \quad 0.5 \leq RIR_{\text{avg}} < 1.5 \\ w_{\text{current}} \cdot (1 - \beta) & \text{if } RIR_{\text{avg}} < 0.5 \quad \text{(deload or excessive fatigue)} \end{cases} \tag{8}$$

where Δw is the standard weight increment ($2.5\,kg$ for compound lifts, $1.25\,kg$ for isolation exercises) and $\beta \in [0.05, 0.10]$ is the fatigue deload factor.

5.4 Gamification Strength Rank Formulation

To provide offline gamification without server dependencies, we calculate a normalized strength rank score S_{rank} based on relative bodyweight performance:

$$S_{\text{rank}} = \left(\frac{1RM_{\text{est}}}{BW} \right) \cdot \lambda_{\text{sex}} \cdot \mu_{\text{type}} \tag{9}$$

where BW represents body weight in kg, $\lambda_{\text{sex}} = 1.45$ for female athletes (1.0 for male), and $\mu_{\text{type}} = 2.2$ for isolation exercises (1.0 for compound lifts). The athlete's strength rank tier (Wood, Iron, Bronze, Silver, Gold, Platinum, Diamond) is assigned by mapped score thresholds $S_{\text{rank}} \in [0.0, 2.5]$.

6 UI Architecture and Design System (Apple Fitness & Glassmorphism Specification)

6.1 Design System Philosophy and OLED Aesthetics

The user interface layer of **FitTracker** transitions from generic UI patterns to an executive, high-performance visual system modeled after the **Apple iOS Fitness** design guidelines and standardized through **shadcn/ui** design token principles.

The UI architecture adheres to four primary design standards:

1. **True OLED Pitch Black Base (#000000)**: Minimizes power consumption on OLED mobile displays while establishing high metric contrast ratio ($> 15:1$).
2. **Squircle Widget Containers (#1C1C1E)**: Rounded rectangular widget containers ($r =$ 20–26px) with subtle border delimiters (#2C2C2E) providing visual hierarchy.
3. **Neon Functional Accent Palette**: Visual telemetry using vivid neon accents:
   - **Neon Pink / Red (#FF2D55)**: Total Volume / Tonnage target.
   - **Neon Green (#30D158)**: Session duration, rest timer completion, and active states.
   - **Neon Cyan (#64D2FF)**: Autoregulated intensity, RPE, and set volume.
   - **Neon Purple (#BF5AF2)**: Repetition counts and strength records.
4. **Glassmorphism Translucent Floating Navigation**: Floating navigation bar utilizing `expo-blur` `BlurView` with dark frosted glass tint, layered above the active content pager.
5. **Athletic Typography System**: Dual Google Fonts hierarchy featuring **Outfit** (extra bold geometric sans-serif for numerical telemetry and headers) and **Plus Jakarta Sans** (clean, high-legibility sans-serif for labels and athlete reports).

6.2 Design Tokens Specification

Table 3 documents the exact color palette, typography, and radius tokens implemented in `src/core/theme/colors.ts`.

6.3 Cross-Platform Navigation and Safe Area Insets

To guarantee zero UI clipping against device physical notches, camera punch-holes, and gesture navigation bars across Android and iOS platforms:

- **Top Status Bar Inset**: Dynamically computed via `StatusBar.currentHeight` with a top offset of 12–16px.
- **Bottom Gesture Inset**: Positioned with an explicit elevation level (elevation $= 10$, $zIndex =$ 9999) and bottom offset (16–24px) preventing collision with Android three-button or gesture pill navigation.
- **Horizontal Gesture Pager**: Implemented using a native `ScrollView` with `pagingEnabled` and `onMomentumScrollEnd` event listeners, allowing seamless bidirectional swiping synchronized with the floating bottom tab bar.

Table 3: FitTracker Design Tokens Specification

Token Identifier	Value / Font Family	Design System Function
`background`	`#000000`	Primary OLED canvas background.
`surface`	`#1C1C1E`	Standard widget container background.
`surfaceLight`	`#2C2C2E`	Elevated cards, inputs, and active glass pills.
`border`	`#38383A`	Structural card borders and dividers.
`primary`	`#30D158`	Apple Fitness Neon Green (Workout / Active).
`secondary`	`#FF2D55`	Apple Fitness Neon Pink/Red (Volume / Move).
`cyan`	`#64D2FF`	Apple Fitness Neon Cyan (RPE / Sets).
`purple`	`#BF5AF2`	Apple Fitness Neon Purple (Reps / Stats).
`headingBold`	`Outfit_800ExtraBold`	Geometric heading & numerical telemetry font.
`bodyRegular`	`PlusJakartaSans_400Regular`	Clean readable body text font.
`glassBackground`	`rgba(28,28,30,0.94)`	High-contrast frosted glass backdrop.
`glassBorder`	`rgba(255,255,255,0.18)`	Glass edge light reflection border.

6.4 FitMojis Avatar System and Launcher Icon Architecture

FitTracker incorporates a custom visual avatar identity system (**FitMojis**) to personalize athlete profiles while maintaining a high-end, minimal 3D design language:

1. **FitMojis Character Set**: Four high-resolution 3D minimal animal head assets (Lion, Bear, Panther, Eagle) sculpted with smooth clay/plastic textures, solid dark backdrops, and subtle athletic accessories (headbands, glasses, headphones).

2. **Mandatory Onboarding Selection & Random Fallback**: Profile creation enforces an explicit avatar selection modal. If an athlete bypasses avatar selection, the system automatically assigns a random FitMoji key (`lion`, `bear`, `panther`, `eagle`) to guarantee visual integrity across athlete cards.

3. **Android Launcher Icon Branding**: Standardized 1024×1024px squircle app launcher icon featuring a matte dark charcoal background (`#18181A`), a central matte cyan ring with a white bold **"FT"** monogram emblem, and a symmetrical 4-quadrant placement of the FitMojis.

7 Session Features, Advanced Analytics, and Streak Buffer Algorithm

7.1 Consistency Streak Buffer Algorithm

To maintain athlete motivation without penalizing planned rest days or micro-deloads, FitTracker implements a rest-tolerant streak calculation algorithm. Let $D = \{d_1, d_2, \ldots, d_n\}$ be the ordered set of unique training dates in epoch milliseconds, sorted in ascending order.

The algorithm defines the inter-session gap in days between consecutive workouts d_i and d_{i+1} as:

$$\Delta d_i = \left\lfloor \frac{d_{i+1} - d_i}{86,400,000} \right\rfloor \tag{10}$$

A streak continuity condition allows up to a maximum rest buffer of $K = 3$ consecutive days:

$$\text{StreakContinuation}(d_i, d_{i+1}) = \begin{cases} \text{True}, & \text{if } \Delta d_i \leq K + 1 \\ \text{False}, & \text{if } \Delta d_i > K + 1 \end{cases} \tag{11}$$

If $\Delta d_i > 4$, the current streak counter resets to 1, accurately balancing discipline tracking with physical recovery.

7.2 Warmup vs. Effective Set Classification and Volume Aggregation

Each recorded set $s \in S$ contains a boolean indicator $is\_warmup \in \{0, 1\}$. Total workout volume V_{total} is computed strictly over effective sets to avoid skewing progressive overload metrics:

$$V_{total} = \sum_{s \in S,\, is\_warmup=0} (w_s \cdot r_s) \tag{12}$$

Where w_s is the weight in kilograms and r_s is the number of completed repetitions. Warmup sets are tagged with custom UI chips (`[Calent.]` / `[Efectiva]`) and excluded from volume PR aggregations.

7.3 Rest Duration Persistence in SQLite Schema

The database schema for `exercise_sets` incorporates a dedicated column `rest_seconds` to persist intra-set rest intervals.

```
CREATE TABLE IF NOT EXISTS exercise_sets (
  id TEXT PRIMARY KEY NOT NULL,
  session_id TEXT NOT NULL,
  exercise_id TEXT NOT NULL,
  set_order INTEGER NOT NULL,
  weight_kg REAL NOT NULL,
  reps INTEGER NOT NULL,
  rpe REAL NOT NULL,
  is_warmup INTEGER NOT NULL DEFAULT 0,
  estimated_1rm REAL NOT NULL,
  rest_seconds INTEGER NOT NULL DEFAULT 0,
  FOREIGN KEY (session_id) REFERENCES workout_sessions (id) ON DELETE CASCADE,
  FOREIGN KEY (exercise_id) REFERENCES exercises (id) ON DELETE CASCADE
);
```

Listing 2: Extended SQLite Table Schema for Exercise Sets

7.4 Pure Line Chart Progression Renderer

The progression module provides dynamic filtering via exercise chips (*Squat, Hip Thrust, Bench Press, Pull-ups, Barbell Row*). The chart calculates normalized 2D coordinate positions (x_i, y_i) for any given dataset of length N:

$$x_i = x_{min} + i \cdot \frac{W - 2 \cdot x_{min}}{N - 1} \tag{13}$$

$$y_i = H - y_{min} - \frac{v_i - v_{min}}{\max(v_{max} - v_{min}, 1)} \cdot (H - 2 \cdot y_{min}) \tag{14}$$

Segment vectors between adjacent points (x_i, y_i) and (x_{i+1}, y_{i+1}) are rendered natively with zero external Metro module resolution overhead using linear transformation matrices:

$$L = \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}, \quad \theta = \text{atan2}(y_{i+1} - y_i,\, x_{i+1} - x_i) \tag{15}$$

7.5 Background Notification Service and Automatic Overload Pre-filling

Using `expo-notifications`, active workout timer states, current exercise names, and rest countdowns synchronize with local push notifications when the application enters background state.

Simultaneously, the progressive overload engine queries historical performance records and automatically pre-fills set rows with calculated target loads (w_{rec}) and target repetitions (r_{rec}), streamlining session execution.

7.6 Internationalization (i18n) and Local-First Privacy Architecture

1. **Dual-Language Localization (ES / EN)**: Centralized i18n dictionary mapping tab titles, workout control buttons, metric telemetry, exercise catalog muscle groups, and FitMoji character labels seamlessly. Athletes select their preferred language during onboarding or toggle it anytime from the profile management widget.

2. **100% Local-First Privacy Guarantee**: Zero external telemetry or server tracking. All training logs, sets, body metrics, and personal notes reside exclusively inside the encrypted device-local SQLite database.

3. **Data Sovereignty & Export**: Full JSON data export capabilities accessible from the session history view, providing athletes complete ownership over their training telemetry.

8 System Verification, Automated Testing, and QA

8.1 Unit Testing Strategy and Test Harness

To ensure deterministic execution of core mathematical overload algorithms, streak calculations, and database use cases, **FitTracker** incorporates a comprehensive automated test suite powered by **Jest** and **TypeScript**.

The test suite covers four architectural domains:

1. **Mathematical Progression Calculators**: Verification of Epley 1RM estimations, Brzycki projections, and volume aggregations.
2. **Strength Ranks Domain Rules**: Classification of 1RM values into strength rank tiers (*Beginner, Intermediate, Advanced, Elite*) across squat, bench press, deadlift, overhead press, and hip thrust.
3. **Streak Buffer Logic**: Automated verification of the 3-day rest allowance rule.
4. **Use Case Orchestrators**: Execution of session creation, set logging with RPE/RIR, catalog filtering, and session finalization.

8.2 Automated Test Suite Summary

Table 4 details the 9 test suites and 33 unit test specs executing cleanly across the codebase.

Table 4: Automated Test Suite Execution Matrix

Test Suite File	Target Domain / Use Case Module	Tests	Status
`strength-ranks.test.ts`	Strength rank classification & 1RM boundaries	5	**PASS**
`calculators.test.ts`	Epley 1RM math & volume calculation formulas	4	**PASS**
`create-workout-session.test.ts`	Session initialization & unique ID assignment	3	**PASS**
`search-exercises.test.ts`	Catalog substring search & muscle group filters	4	**PASS**
`streak.test.ts`	Rest-tolerant 3-day streak buffer algorithm	4	**PASS**
`get-exercise-history.test.ts`	Time-series aggregation & 1RM history generation	3	**PASS**
`register-set.test.ts`	Live set logging, warmup tagging, RPE validation	4	**PASS**
`finish-workout-session.test.ts`	Session finalization, duration, and PR detection	3	**PASS**
`get-progression-recommendation.test.ts`	Autoregulated progressive overload rules	3	**PASS**
Total Execution Metrics	**9 Test Suites**	**33 / 33**	**100% PASS**

8.3 Manual QA and User Acceptance Testing (UAT)

In addition to automated unit tests, end-to-end manual QA validation was performed on physical Android and iOS devices using Expo Go (SDK 54):

- **Background State Persistence**: Verified that background timer notifications update continuously when navigating away from the active workout screen.

- **Gesture & Orientation Ergonomics**: Validated smooth horizontal swiping between tab screens with zero visual overlap on device status bars or gesture navigation pills.
- **FitMojis UI Rendering**: Verified seamless rendering of 3D minimal animal avatar cards across profile setup and avatar picker modals.

9 Conclusion and Future Engineering Roadmap

9.1 Conclusion

FitTracker successfully delivers a high-performance, executive mobile solution for resistance training management built on a strict **Local-First** architecture [3]. By integrating validated progressive overload mathematical models [4, 5] and RPE/RIR autoregulation principles [6, 7], the system eliminates manual calculation friction for strength athletes.

The combination of an Apple Fitness-inspired OLED pitch black design system, custom **FitMojis** visual identity, background workout telemetry notifications, dynamic rest-tolerant streak calculations, and dual-language localization (ES/EN) ensures a premium user experience with complete data sovereignty and zero cloud dependency.

9.2 Production Deployment Readiness

The application is fully prepared for binary production build generation (`.apk` / `.aab` for Android Google Play Store and `.ipa` for Apple App Store) using Expo Application Services (EAS Build) on Expo SDK 54 [2].

Key production milestones completed:

- **ACID SQLite Engine**: Embedded local database schema with automatic table migrations.
- **Standardized Launcher Assets**: High-resolution squircle app launcher icon (`assets/icon.png`) featuring the FitMojis composition and matte cyan "FT" monogram.
- **Full Test Coverage**: 100% passing automated test suites (33/33 test specs).

9.3 Future Engineering Roadmap

Potential future extensions for subsequent releases include:

1. **Bluetooth LE Sensor Integration**: Real-time velocity-based training (VBT) encoder telemetry via Bluetooth Low Energy.
2. **Cloud Sync Peer-to-Peer Backup**: End-to-end encrypted peer-to-peer synchronization across multiple personal devices without centralized user tracking.
3. **Advanced Biometric Wearable Sync**: Integration with Apple HealthKit and Google Health Connect for automatic heart rate and calorie expenditure correlation.

References

[1] Meta Open Source, "React native: Learn once, write anywhere," https://reactnative.dev/, 2024.

[2] Expo Documentation, "Expo sdk 54 framework specification and native modules," https://docs.expo.dev/, 2024.

[3] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," *Proceedings of the ACM on Programming Languages*, vol. 3, no. ONSET, pp. 1–24, 2019.

[4] B. Epley, "Poundage chart," *Boyd Epley Workout*, pp. 14–15, 1985.

[5] M. Brzycki, "Strength testing—predicting a one-rep max from repetitions to fatigue," *Journal of Physical Education, Recreation & Dance*, vol. 64, no. 1, pp. 88–90, 1993.

[6] M. C. Zourdos, A. Klemp, C. Dolan, J. M. Quiles, K. A. Schau, E. Jo, E. Helms, B. Esgro, S. Duncan, S. Garcia, and M. A. Naimo, "Novel resistance training rating of perceived exertion scale measuring repetitions in reserve," *The Journal of Strength & Conditioning Research*, vol. 30, no. 1, pp. 267–275, 2016.

[7] E. R. Helms, J. Cronin, A. Storey, and M. C. Zourdos, "Application of the repetitions in reserve-based rating of perceived exertion scale for resistance training," *Strength and Conditioning Journal*, vol. 38, no. 4, pp. 42–49, 2016.

[8] IEEE Computer Society, "Ieee recommended practice for software requirements specifications," 1998.